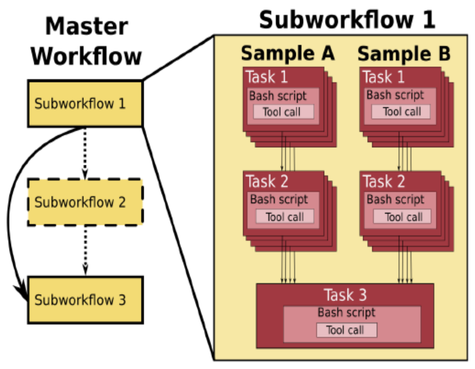
Comparative analysis of workflow management systems in production genomics research and the clinic

AE Ahmed[1,2*], J Allen[3], S Baheti[4], T Bhat[3,5,6], MA Bockol[7], P Burra[3,5], TM Drucker[7], SNHart[8], JR Heldenbrand[3], ME Hudson[3,9], MT Kalmbach[7], G Kapraun[7], EW Klee[8], K Kendig[3], M Kendzior[3], NR Mattson[7], CA Ross[7], SM Sharif[2], ED Wieben[10], M Wiepert[7], RVenkatakrishnan[3], FM Fadlelmola[1†], and LS Mainzer [3,11†+]

1 Introduction

Today's era of data intensive science is introducing drastic changes to the scientific method of conducting research, manifest as increased reliance on computation. In bioinformatics applications today, routine analyses tend to embed multiple levels of complexity, and that constraints workflow design.



On the one hand, this warrants modular workflow construction for easier abstraction and reuse (R in FAIR code). On the other, this logic needs to be programmatically expressed in an efficient manner. To satisfy these requirements, the bioinformatics community has largely settled on CWL, WDL and Nextflow as workflow languages. We systematically examine their features and reflect on the aspects that could drive the choice of a workflow management system for a given application. We also consider the effects of computational ecosystem and community, and the needs of large scale collaborations, production laboratories, and clinical settings. With ongoing innovations in technology, the needs of scientific communities evolve. Thus, workflow management systems will affect and be affected by progress at the levels of infrastructure, programming, and user interface.

2 Methods

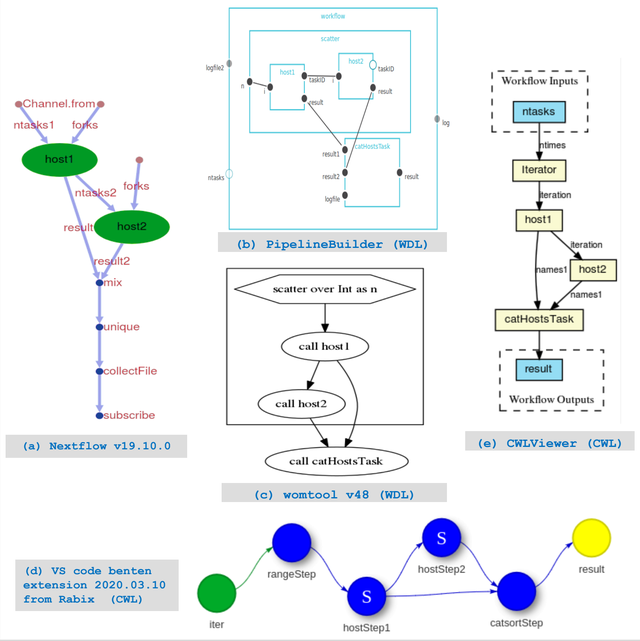
We performed a systematic evaluation of popular bioinformatics Workflow Management Systems (WfMS) and some of their executors. As a counterpoint we also included Swift/T, a WfMS commonly used in high scale physics applications as in the table below. Two example workflows were implemented and run locally, on an HPC cluster and in the cloud, to perform an in-depth comparison in terms of language expressiveness, modularity, scalability, robustness, reproducibility, interoperability, ease of development, and adoptability.

WfMS Language	WfMS Engine	Case Study 1: Variant calling pipeline	Case Study 2: Scalability evaluation
swift		GATK3; multi-sample; single-step if needed (Ahmed et al., 2019); https://swift-t-variant-calling.readthedocs.io/en/latest/	-
nextflow		GATK 4; multi-sample; https://github.com/ncsa/Genomics_MGC_VariantCalling_Nextflow/tree/dev-gatk†	Same repository for these three WfMSs (https://github.com/azzaea/scalability-tst)
COMMON WORKFLOW LANGUAGE		-	
{wdl}		GATK4; single sample; https://github.com/ncsa/MayomicsVC/tree/dev-gatk†	

3 Language features comparison

Aspect	nextflow	swift	COMMON WORKFLOW LANGUAGE	{wdl}
Parent language	Ruby and Groovy	C, tcl	N/A	N/A
GUIs	NextflowWorkbench (Kurs et al., 2016), DolphinNext (Yukselen et al., 2019)	-	Rabix composer	Pipeline Builder (EPAM systems, 2019)
DSL features	complete, extensible in Groovy and Java	complete, extensible in tcl	limited standard library, extensible via javascript	limited standard library
Variables	qualified, unique within scope	typed, unique within scope	typed, unique identifiers	typed, fully qualified names
Loops	parallel queue channels	sequential for and parallel foreach	parallel scatter via ScatterFeatureRequirement	parallel scatter
Conditionals	via when declaration within a process	if-else and no-fall through switch statements	when and pickValue fields proposed in CWLv1.2	if blocks producing optional output types
Enforcing good practices	nf-core (Ewels et al., 2019)	-	CWL guide (Hodges and Crusoe, 2020)	-

DAGs of the *scalability-evaluation* case study



Engine features comparisons

WfMS Language	WfMS Execution engine	Remarks
nextflow, swift		Complete WfMS, supporting conditionals, loops and nested logic.
COMMON WORKFLOW LANGUAGE	*cwltool*	The official reference implementation of execution engine for the complete CWL standard (CWL group, 2020b); no cluster or cloud support.
	cromwell (1.0) toil (1.0.1)	Supported via WOM. Optimized for cloud environments.
	rabix executor (sbg:draft-2, 1.0)	This single node local executor is no longer supported by the original developer team at Seven Bridges.
	arvados, cwl-tes, airflow	Tedious setup.
{wdl}	*cromwell* (draft-2, 1.0)	De facto standard for executing WDL workflows. Support for nested loops is version-dependent.
	toil-wdl-runner (draft-2) miniWDL (draft-2, 1.0)	No support for modularity or nesting of loops and conditionals. No cluster or cloud support. Includes Cromwell wrapper.

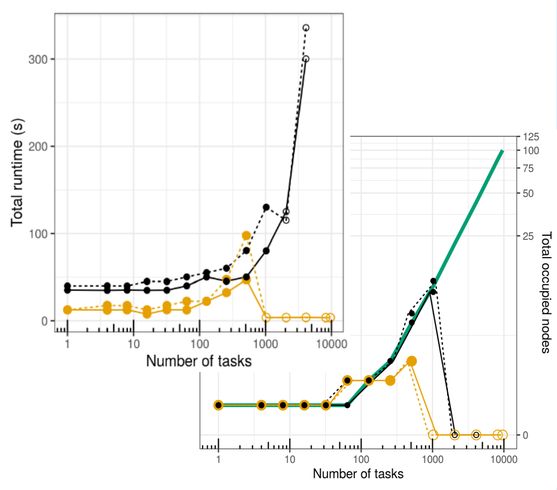
WfMS Language	WfMS Engine	Biocluster	AWS Batch mode	AWS Cloud cluster	AWS Parallel Cluster
Swift/T		I, II	-	-	-
Nextflow		I, II	-	I	II
CWL	cromwell	I, II	-	-	-
WDL	cromwell	I, II	I	-	II

Biocluster is a Slurm-based HPC. We used the Normal queue of five Supermicro SYS-2049U-TR4 nodes of 72 Intel Xeon Gold 6150 2.7 GHz cores, 1.2 TB RAM each.
The ***AWS Parallel-cluster*** is composed of 100 m5a.24xlarge instances with 96 cores and ~412 GB RAM each

4 Scalability evaluation

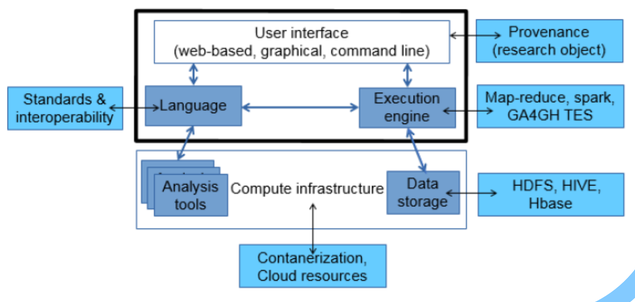
The workflows in the DAGs above were deployed on AWS parallel cluster in order to evaluate: (1) How well does the execution engine scale with the number of parallel tasks? (2) How well does the engine pack tasks on a node?

The figures to the right show the scaling of a one-step (solid line) and two-step (dashed line) workflows in Cromwell+WDL (black) and Nextflow (yellow). The thick green line in the right panel is the theoretical optimum of the number of nodes that should be occupied by the tasks, computed as the ceiling of tasks/cores per node. Empty circles denote failed runs.



5 Reflections

- Nextflow, CWL and WDL, unlike Swift/T, were produced by practicing bioinformaticians openly addressing their own analysis needs, such as dynamic scheduling, portability, robustness and accessibility.
- Clinical settings benefit from those aspects, with additional focus on validation and conformance to regulations.
- The wide adoption of these WfMSs is possible due to permissive open source licenses used: Apache 2.0 for CWL and Nextflow, BSD 3 Clause for WDL. Open sharing practices established by the FAIR guidelines and publicity in large scale consortia and driver projects (e.g. GA4GH) also help with adoption. Industry has been both a player and a shareholder in this arena.
- The rapid rate of evolution of the open sharing standards necessitates frameworks for evaluating conformance of the workflow execution engine.
- Infrastructure, big data analytics, and transparency will continue to be areas of innovation.



Find our works!

[1] Ahmed AE, Heldenbrand J, et al. (2019) Managing genomic variant calling workflows with Swift/T. PLOS ONE 14(7): e0211608. https://doi.org/10.1371/journal.pone.0211608

[2] Mainzer LS, Balgi P, et al. (2020) ncsa/MayomicsVC v1.0.0. Zenodo https://doi.org/10.5281/zenodo.3679275

[3] Ahmed AE, Allen J et al, Requirements-driven design considerations for workflow management systems used in production genomics research and the clinic. (In process)

Acknowledgments

This work was a product of the Mayo Clinic and Illinois Alliance for Technology-Based Healthcare. Special thanks for the funding provided by the Mayo Clinic Center for Individualized Medicine and the Todd and Karen Wanek Program for Hypoplastic Left Heart Syndrome. We also thank the Interdisciplinary Health Sciences Institute, the Carl R. Woese Institute for Genomic Biology and the National Center for Supercomputing Applications for their generous support and access to resources. We particularly acknowledge the support of Keith Stewart, M.B., Ch.B., Mayo Clinic/Illinois Grand Challenge Sponsor and Director of the Mayo Clinic Center for Individualized Medicine. Special gratitude to Gay Reed and Amy Weckle for managing the project. Many thanks to the Biocluster team for their consultation and advice during the deployment of our workflows on their machine. Thanks also to the UIUC AWS infrastructure, and the AWS Research Credits Award for supporting this work. Finally we are grateful for the support of H3ABioNet, funded by the National Institutes of Health Common Fund under grant number U41HG006941.

Contact information

* Bernoulli Institute, University of Groningen, Netherlands
: azzaea@gmail.com ; : @azzaea_